

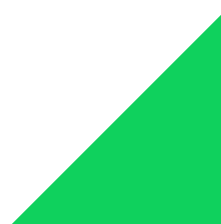
Kubectl 编程模式之Builder和Visitor



内容大纲

1. Builder模式

2. Visitor模式



Builder模式（构造方法）

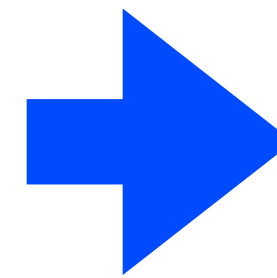
最简单的构造函数

```
// 默认构造函数  
p, _ := rocketmq.NewProducer()  
  
// 带有Timeout参数的构造函数  
p, _ := rocketmq.NewProducerWithTimeout(60*time.Second)
```

缺点：扩展性不够

带配置参数的构造函数:

```
type Config struct {  
    Timeout *time.Time  
    Retry int  
}  
  
p, _ := rocketmq.NewProducer(&config)
```

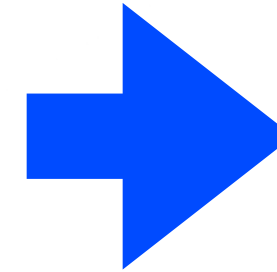


```
func NewProducer(c *Config) *Producer {  
    p := &Producer{}  
    if c.Timeout != nil {  
        p.Timeout = c.Timeout  
    }  
  
    if c.Retry > 0 {  
        p.Retry = c.Retry  
    }  
  
    return p  
}
```

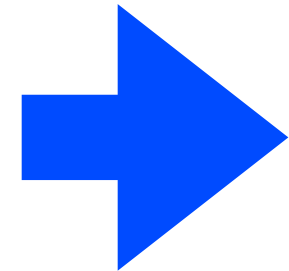
缺点：不够简洁，需要判断参数值

通过闭包实现可变长参数的构造函数:

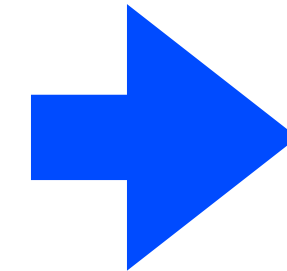
```
sched, err := scheduler.New(  
    cc.Client,  
    // ...  
    scheduler.WithKubeConfig(cc.KubeConfig),  
    scheduler.WithProfiles(cc.ComponentConfig.Profiles...),  
    // ...  
)
```



```
// 返回一个闭包函数  
func WithKubeConfig(cfg *restclient.Config) Option {  
    return func(o *schedulerOptions) {  
        o.kubeConfig = cfg  
    }  
}
```



```
func New(client clientset.Interface, opts ...Option) (*Scheduler, error) {  
    // ...  
    options := defaultSchedulerOptions  
    for _, opt := range opts {  
        opt(&options) // 将 New 方法传入的参数给了options  
    }  
    // ...  
}
```

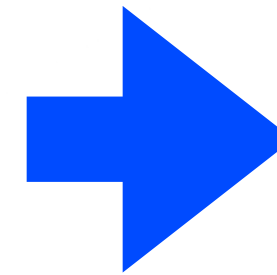


```
configurator := &Configurator{  
    componentConfigVersion: options.componentConfigVersion,  
    percentageOfNodesToScore: options.percentageOfNodesToScore,  
    podInitialBackoffSeconds: options.podInitialBackoffSeconds,  
    podMaxBackoffSeconds: options.podMaxBackoffSeconds,  
    extenders: options.extenders,  
    frameworkCatcher: options.frameworkCatcher,  
    parallelism: options.parallelism,  
}
```

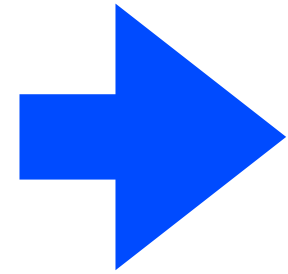
缺点: 传入参数过多, 不够优雅。

Builder模式：

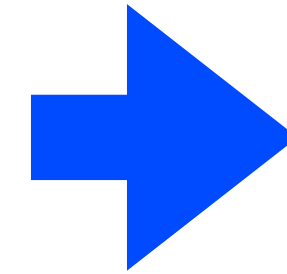
```
r := f.NewBuilder().  
    // ...  
    RequestChunksOf(chunkSize).  
    Latest().  
    Do()
```



```
// 返回 Builder 实例  
NewBuilder() *resource.Builder
```

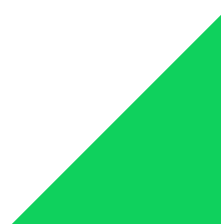


```
// 更新 Builder 实例的 limitChunks  
func (b *Builder) RequestChunksOf(chunkSize int64) *Builder {  
    b.limitChunks = chunkSize  
    return b  
}
```



```
// 更新 Builder 实例的 latest值  
func (b *Builder) Latest() *Builder {  
    b.latest = true  
    return b  
}
```

优雅且易读。设置好默认参数，通过链式调用动态配置需要修改的参数。



Visitor模式(链式调用)

结构体方法链式调用：

```
person.SetAge(10).SetName("kobe").SetGender("male")
```

循环调用：

```
http.Handler("/apiserver", authn, authz, admission)

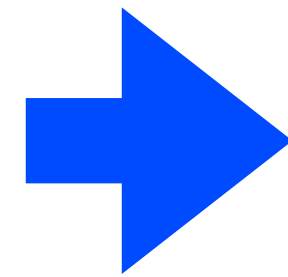
for i := range methods {
    methods[i].call()
}
```

使用Channel:

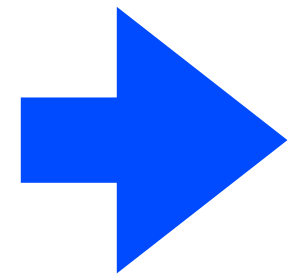
```
for {
    select {
        case <- ctx.Done():
            return
        default:
            method <- methodsCh
            method.call()
    }
}
```


Visitor模式 - 定义:

```
// Visitor 是带有 Visit 方法的接口
type Visitor interface {
    Visit(VisitorFunc) error
}
```



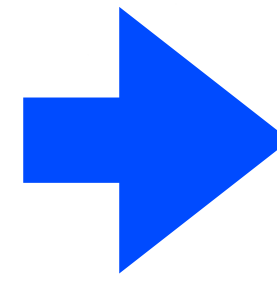
```
// 一个实现了 Visitor 接口的结构体定义, 需要有 visitor 字段
type DecoratedVisitor struct {
    visitor    Visitor
    decorators []VisitorFunc
}
```



```
// Visit 方法最关键的在于, 它会调用自己 `visitor` 的 `Visit` 方法, 形成调用链。
func (v DecoratedVisitor) Visit(fn VisitorFunc) error {
    return v.visitor.Visit(func(info *Info, err error) error {
        if err != nil {
            return err
        }
        // ...
        return fn(info, nil)
    })
}
```


Visitor模式 - 声明:

```
// builder 模式构造函数
r := f.NewBuilder().
    // ...
    RequestChunksOf(chunkSize).
    Latest().
    Do()
```



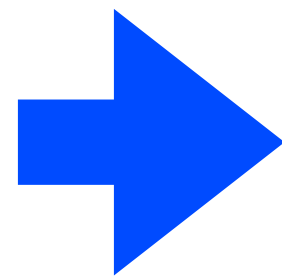
```
func (b *Builder) Do() *Result {
    r := b.visitorResult()

    // ...

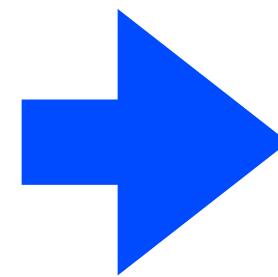
    r.visitor = NewFlattenListVisitor(r.visitor, b.objectTyper, b.mapper)

    // ...
    r.visitor = NewDecoratedVisitor(r.visitor, helpers...)

    return r
}
```



```
// Result 结构体
type Result struct {
    visitor Visitor
    // ...
}
```

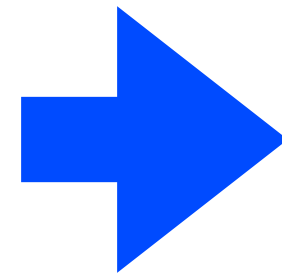


```
// Result 就是实现 Visitor 接口的结构体
func (r *Result) Visit(fn VisitorFunc) error {
    if r.err != nil {
        return r.err
    }
    err := r.visitor.Visit(fn)
    return utilerrors.FilterOut(err, r.ignoreErrors...)
}
```

返回了一个 Visitor 多层嵌套的 Result 实例: Result.Visitor -> NewDecoratedVisitor(NewFlattenListVisitor(Visitor))

Visitor模式 - 声明:

```
infos, err := r.Infos()
```

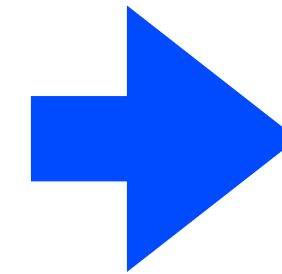


```
func (r *Result) Infos() ([]*Info, error) {  
    // ...  
  
    infos := []*Info{}  
    err := r.visitor.Visit(func(info *Info, err error) error {  
        if err != nil {  
            return err  
        }  
        infos = append(infos, info)  
        return nil  
    })  
  
    // ...  
    return infos, err  
}
```

调用 Result 实例的 Visit() 方法, 实现类似深度遍历搜索(DFS)的方法调用: Visiotr.Visit() -
> NewFlattenListVisitor.Visit() -> NewDecoratedVisitor.Visit()

Visitor模式 - 注意:

```
func (v Num<num>Visitor) Visit(fn VisitorFunc) error {  
    return v.visitor.Visit(func(info *Info, err error) error {  
        fmt.Println("before function call-<num>")  
        err = fn(info, err)  
        if err == nil {  
            fmt.Println(err)  
        }  
        fmt.Println("after function call-<num>")  
        return err  
    })  
}
```



```
before function call-1  
before function call-2  
before function call-3  
...  
after function call-3  
after function call-2  
after function call-1
```


Visitor模式 - 总结：

Visitor模式其实就是一颗单分支的 DFS 遍历树。

THANKS



DaoCloud

驾驭数字方程式
THE FORMULA DIGITAL